

1. プロジェクト概要

Questra は、「やるべきこと」をクエストとして完了させることで、画面上の植物を育て、卵を発見・孵化させてドラゴンを仲間にする、ゲーミフィケーション型タスク管理アプリケーションです。

コアコンセプト

タスク完了 → EXP/コイン獲得 → 植物成長
継続するほど「元気」が溜まり、卵を発見
卵が孵化 → ドラゴンが仲間に
4人の「植物先生」が性格に合わせてサポート

キー特徴

バックエンド不要・localStorage 完結
プライバシー・コスト・運用すべてゼロ
モダンでミニマルな UI/UX
TypeScript strict モードで型安全性確保

対応環境

モバイルファースト（親指操作最適化）
PWA 対応予定（インストール可能）
オフライン動作（Service Worker 予定）
モダンブラウザ全対応

2. 技術スタック

カテゴリ	技術 / ライブラリ	バージョン / 備考
フレームワーク	React	18.3.1
言語	TypeScript	5.5.x (strict mode)
ビルドツール	Vite	5.4.x
スタイリング	Tailwind CSS	3.4.x (カスタムテーマ)
状態管理	React Hooks + localStorage	単一フック usePlantData
永続化	localStorage	キー: questra_plant_data
デプロイ	Cloudflare Pages	静的ホスティング、エッジ配信
リポジトリ	GitHub	ryotagtagtag-wq/schoolsync
本番 URL	Custom Domain	https://quest.ryopc.org
画像ホスティング	ImageKit.io	卵・ドラゴン画像の配信

主要依存関係 (package.json 抜粋)

```
{
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@types/react": "^18.3.3",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.1",
    "autoprefixer": "^10.4.19",
    "postcss": "^8.4.38",
    "tailwindcss": "^3.4.4",
    "typescript": "^5.5.3",
    "vite": "^5.4.11"
  }
}
```

```
}  
}
```

3. ファイル構成

```
schoolsync/  
├── index.html          # エントリーポイント  
├── package.json        # 依存関係・スクリプト  
├── tsconfig.json       # TypeScript 設定 (strict)  
├── tsconfig.node.json  # Node 用 TS 設定  
├── vite.config.ts      # Vite 設定  
├── tailwind.config.js  # Tailwind カスタムテーマ  
├── postcss.config.js   # PostCSS 設定  
├── SPEC.html           # 本仕様書  
├── SPEC.md             # Markdown 版仕様書  
└── src/  
    ├── main.tsx        # アプリエントリー  
    ├── index.css        # グローバルスタイル + Tailwind directives  
    ├── types/index.ts   # 型定義・定数 (PLANT_TYPES, SHOP_ITEMS 等)  
    ├── utils/plantLogic.ts # 植物成長ロジック・元気計算  
    ├── hooks/usePlantData.ts # 状態管理フック (核心)  
    ├── data/teacherLines.ts # 植物先生セリフデータ  
    └── components/  
        ├── App.tsx      # ルート・タブ制御・朝の挨拶表示  
        ├── PlantDisplay.tsx # 植物表示・元気バー・なでなで・鉢ステージ  
        ├── TaskList.tsx  # クエスト追加/削除/完了 UI  
        ├── MorningGreeting.tsx # 朝の挨拶・卵孵化フロー全制御  
        ├── Garden.tsx   # 複数植物管理・名前変更・新規迎え入れ  
        ├── Shop.tsx     # コイン表示・アイテム購入  
        ├── Backpack.tsx # 所持アイテム表示・使用  
        ├── Quest.tsx    # 統計・テンプレート・カテゴリ・作成モダル  
        └── TabBar.tsx   # 固定ボトムナビ (5タブ)
```

4. データモデル

4.1 localStorage スキーマ (questra_plant_data)

```
interface PlantData {
  exp: number; // 総経験値 (植物成長に使用)
  coins: number; // 所持コイン (ショップ購入用)
  tasks: Task[]; // クエスト一覧
  ownedItems: OwnedItem[]; // 購入済みアイテム (アイテムID × 個数)
  plants: UserPlant[]; // 複数植物データ
  activePlantId: string; // 現在表示中の植物ID
  egg: EggData | null; // 卵データ (未発見時は null)
  dragons: Dragon[]; // 孵化済みドラゴン一覧
  lastVitalityDecay: number; // 元気自動減衰最終実行時刻 (ms)
  lastMorningGreeting: string; // 朝の挨拶最終表示日 (YYYY-MM-DD)
  version: number; // データバージョン (移行用)
}
```

4.2 主要型定義 (types/index.ts)

型名	フィールド	説明
Task	id: string	ユニークID (generateId())
	text: string	タスク内容
	completed: boolean	完了フラグ
	createdAt: number	作成時刻 (ms)
	category?: string	カテゴリ (仕事/勉強/健康/家事/その他)
	isTemplate?: boolean	テンプレートフラグ
UserPlant	id: string	植物インスタンスID
	type: PlantType	植物種別 (ひまわり/サボテン/モンスター/タネ)

型名	フィールド	説明	
	nickname: string	ユーザー設定のニックネーム	
	exp: number	この植物の経験値	
	vitality: number	元気値 (0-100, 水やりで回復)	
OwnedItem	itemId: string	SHOP_ITEMS の ID	
	quantity: number	所持数	
		stage: 'hidden' 'found' 'hatching' 'hatched'	卵の状態
progress: number	EggData	孵化進捗 (0-100)	
foundAt: number		発見時刻	
discoveredBy: PlantType		発見した植物先生	
Dragon		id: string	ドラゴン ID
	name: string	ユーザー命名の名前	
	hatchedAt: number	孵化時刻	
	imageUrl: string	ドラゴン画像 URL	
	parentPlantType: PlantType	親植物の種類	

4.3 植物種類定数 (PLANT_TYPES)

ID	名前	絵文字	鉢植え絵文字	花絵文字	説明
sunflower	ひまわり				元気いっぱい、太陽が好き
cactus	サボテン				マイペース、水は控えめでOK

ID	名前	絵文字	鉢植え絵文字	花絵文字	説明
monstera	モンステラ	🌿	🪴	🌸	優雅、大きな葉が特徴
seed	タネ	🌰	🌱	🌺	何にでもなれる可能性の塊

4.4 ショップアイテム (SHOP_ITEMS)

ID	名前	絵文字	価格(コイン)	効果	説明
water	まほうの水	💧	30	vitality +30	植物の元気を回復させる
sunlight	ポカポカ光	☀️	50	vitality +50	太陽の光で元気いっぱい
fertilizer	栄養たっぷり肥料	🌿	100	vitality +80, exp +20	成長も促すスペシャルアイテム

5. 機能仕様

5.1 植物成長システム

成長段階 (経験値ベース・線形)

Stage 0 0-49 EXP - ふたば 🌱

Stage 1 50-99 EXP - わかば 🌿

Stage 2 100-199 EXP - お気に入りの鉢植え 🪴

Stage 3 200+ EXP - きれいなお花 🌸

経験値獲得ルール

- ✅ タスク完了: +10 EXP / +10 コイン
- 🔄 完了取り消し: -10 EXP / -10 コイン (最小値 0)
- 🌿 肥料使用: +20 EXP + 元気 +80
- 📊 進捗バー: 次の段階までの残り EXP をリアルタイム表示

元気システム

- 💧 水やり忘れで自動減衰: 1分ごとに -1 (usePlantData で setInterval)
- 🔴 最小値 0 (枯れない設計)
- 📊 元気バーで視覚化 (PlantDisplay に表示)
- 💧 アイテム使用で即時回復

5.2 植物先生システム (4キャラクター)

各植物種類に専属の「先生」がおり、性格に合わせたセリフでユーザーをサポートします。

🌻 ひまわり先生

元気でポジティブ。太陽のような明るさで励ます。「今日もがんばろー！」

サボテン先生

ツンデレ・マイペース。厳しめだが根は優しい。「ま、やるなら見ててやるよ」

モンステラ先生

優雅で知的。穏やかに見守り、気づきを与える。「ゆっくりでいい、着実にね」

タネ先生

無限の可能性。好奇心旺盛でワクワクさせる。「なにができるかな？たのしみだね！」

セリフカテゴリ (teacherLines.ts)

カテゴリ	発火タイミング	用途
morning	アプリ起動時 (1日1回、朝 5:00-11:59)	朝の挨拶モーダル表示
afterComplete	タスク完了時	完了時の励まし・卵発見トリガー
eggFound	卵発見時 (25%確率)	「卵見つけたよ！」報告
eggGrowing	卵成長中 (タスク完了時)	「もうすぐ生まれそう」経過報告
dragonBorn	孵化完了時	「生まれた！」祝福
dragonGreeting	ドラゴン仲間入り後	朝の挨拶でドラゴンも登場

5.3 卵・ドラゴンシステム

卵発見～孵化フロー

タスク完了 → 25% の確率で卵発見 (eggFound セリフ発火)

見守りフェーズ → MorningGreeting で「見守る」ボタン表示

成長 → 以降のタスク完了ごとに **+15% 進捗** (eggGrowing セリフ)

孵化 (100%) → プロGRESSバー自動アニメーション → dragonBorn セリフ

ドラゴン誕生 → 画像表示 → 名前入力モーダル → 決定で仲間入り

完了 → ドラゴン画像・名前表示 → 以降 dragonGreeting で朝に登場

技術仕様

- 🥚 卵画像: <https://ik.imagekit.io/.../egg.png> (ImageKit.io 配信)
- 🐉 ドラゴン画像: <https://ik.imagekit.io/.../dragon.png> (共通、今後種別拡張予定)
- 🕒 孵化PROGRESSバー: requestAnimationFrame で滑らかな 0→100% 自動進行 (約 3 秒)
- 📝 名前入力: モーダルで入力、空白不可、最大 10 文字、Enter で確定
- 💾 永続化: dragons[] 配列に追加、localStorage 即時保存

5.4 朝の挨拶モーダル (MorningGreeting)

表示条件・制御

- 🕒 時間帯: 朝 5:00～11:59 のみ表示
- 📅 頻度: 1日1回 (lastMorningGreeting で日付管理)
- 🎯 トリガー: App.tsx マウント時に checkAndShowMorningGreeting() 実行
- 🐉 ドラゴン同伴: 孵化済みドラゴンがいる場合、一緒に登場 (dragonGreeting)

モーダル内フロー

- 📖 通常時: 先生の朝の挨拶 (morning) 表示 → 「わかった！」で閉じる
- 🥚 卵発見時: eggFound セリフ → 「見守る」ボタンで孵化フロー開始
- 🕒 孵化中: PROGRESSバー自動進行表示 → 100% で次のステップへ
- 🐉 孵化完了: ドラゴン画像 + dragonBorn → 名前入力モーダルへ遷移
- 📝 名前入力: フォーム入力 → 「名前を決める」で確定 → 完了画面

5.5 ガーデン - 複数植物管理

- 🌿 **植物一覧表示:** 所有植物をカード形式で表示 (種類絵文字・ニックネーム・成長段階・元気バー)
- 🔄 **アクティブ切替:** タップで `activePlantId` 変更、メイン画面の植物が即座に切り替わる
- ✏️ **ニックネーム変更:** 各植物カードでインライン編集可能
- ➕ **新規植物迎え入れ:** 「新しい植物を迎える」ボタン → 種類選択モータル → ニックネーム入力 → `exp: 0, vitality: 100` で追加
- 🗑️ **植物お別れ:** 確認モータル後削除 (最後の1体は削除不可)

5.6 ショップ & バックパック

🛍️ Shop.tsx

- 💰 ヘッダーに所持コイン表示
- 📦 アイテムカード: 絵文字・名前・価格・効果説明・購入ボタン
- 🚫 **購入制御:** コイン不足時はボタン無効化 (`opacity-50, pointer-events-none`)
- ✅ **購入時:** コイン即時減算、`ownedItems` に追加/個数インクリメント、即時保存

🎒 Backpack.tsx

- 📋 所持アイテム一覧 (名前・絵文字・個数・効果)
- 📦 **使用ボタン:** タップで `useItem(itemId)` 実行
- ✨ **使用エフェクト:** `PlantDisplay` で「なでなで」アニメーション (バウンス + キラキラ 2秒) + 「わーい! 💕」ポップイン
- 📄 **使用时:** `ownedItems` 個数デクリメント (0 で削除)、効果即時適用 (`vitality/exp`)、即時保存

5.7 クエスト管理 (Quest.tsx)

- 📊 **統計ダッシュボード:** 総タスク数・完了数・完了率・総獲得 EXP・総コインをカード表示
- ⚡ **テンプレートクイック追加:** 仕事/勉強/健康/家事/その他 カテゴリ別プリセット (各 3-4 件) をワンクリック追加
- 🔍 **カテゴリフィルタ:** 全件/カテゴリ別絞り込みタブ
- ➕ **カスタム作成モータル:** テキスト入力 + カテゴリ選択 → 追加
- 📝 **ルール説明:** アコーディオンで「クエストのルール」表示 (完了で +10 EXP/コイン、取消で -10 等)
- ✅ **完了トグル:** チェックボックスで完了/未完了切替、即時保存・即時反映
- 🗑️ **削除:** ゴミ箱アイコンで削除 (確認なし、即時)

5.8 データ永続化・安全性

保存戦略 (usePlantData.ts)

 **即時保存:** 全状態変更操作後に `saveData()` 呼び出し

 **try-catch ラップ:** 読み書き両方で例外吸収、失敗時はデフォルト値で安全起動

 **ID 生成:** `Date.now().toString(36) + Math.random().toString(36).slice(2)` (衝突耐性)

 **バージョンニング:** `version` フィールドで将来のスキーマ移行に対応

エラーハンドリング

 **プライベートモード対応:** `localStorage` アクセス失敗時もアプリクラッシュせず初期値で起動

 **容量超過吸収:** `QuotaExceededError` も `catch` し、古いデータ削除等のリカバリ余地確保

 **タブ間同期:** 同一オリジンなら `storage` イベントで自動同期 (将来実装)

 **開発時デバッグ:** `localStorage.clear()` で簡単リセット可能

6. UI/UX 仕様

カラーパレット



ベース背景 #ffffff / #0f172a



代替背景 #f8fafc / #1e293b

プライマリ (緑) #059669

プライマリライト #d1fae5

アクセント (青) #2563eb

警告 (琥珀) #d97706

危険 (赤) #dc2626



ボーダー #e2e8f0

タイポグラフィ

見出し: Inter (Weight 600/700)

本文: Inter (Weight 400/500/600)

コード: JetBrains Mono (Weight 400/500)

ベースサイズ: 16px (1rem)

行間: 1.7 (leading-relaxed)

コンポーネント共通スタイル (tailwind.config.js / index.css)

```
/* カスタムコンポーネントクラス */
.btn-primary { @apply bg-primary text-white font-semibold px-6 py-3 rounded-full
  transition-all hover:bg-primary-hover; }
.btn-secondary { @apply bg-white border border-gray-300 text-gray-700 font-semibold |
  transition-all hover:bg-gray-50; }
.input-field { @apply w-full px-4 py-3 border border-gray-300 rounded-lg
  focus:outline-none focus:border-primary focus:ring-2 focus:ring-primary/20; }
.card-base { @apply bg-white border border-gray-200 rounded-xl shadow-sm; }
.modal-overlay { @apply fixed inset-0 bg-black/30 backdrop-blur-sm z-50 flex items-c
```

```
.modal-content { @apply bg-white rounded-xl border border-gray-200 p-6 max-w-md w-full shadow-lg animate-pop-in; }
```

6.1 画面構成・タブナビゲーション

タブ	アイコン	主要コンポーネント	役割
メイン		PlantDisplay, TaskList	植物表示・元気バー・クエスト完了操作
クエスト		Quest	統計・テンプレート・カテゴリ・作成・ルール
ショップ		Shop	コイン確認・アイテム購入
バックパック		Backpack	所持アイテム確認・使用 (エフェクト発火)
ガーデン		Garden	複数植物管理・切替・名前変更・新規迎え入れ

6.2 アニメーション・インタラクション

アニメーション	実装場所	仕様
植物バウンス	PlantDisplay.tsx	<code>animate-bounce-gentle</code> + <code>wiggle</code> (2秒、アイテム使用時)
キラキラエフェクト	PlantDisplay.tsx	8個の ✨ がランダム位置に <code>animate-sparkle</code> (2秒)
ポップイン	PlantDisplay, MorningGreeting	<code>animate-pop-in</code> (scale 0→1, 300ms ease-out)
プログレスバー	PlantDisplay, MorningGreeting	<code>transition-all duration-500 ease-out</code> width 変化
孵化プログレス	MorningGreeting.tsx	<code>requestAnimationFrame</code> で 0→100% (約 3秒、ease-out)
モーダル表示	全モーダル共通	<code>animate-pop-in</code> + オーバーレイフェードイン
タブ切替	TabBar.tsx	インジケータースライド (CSS transition)

6.3 レスポンシブ・アクセシビリティ

- 📱 **モバイルファースト**: 320px〜 対応、固定ボトムバーで親指操作最適化
- 💻 **デスクトップ**: 最大幅 640px で中央寄せ、余白活かしたレイアウト
- 🔊 **ARIA 対応**: `role="img"` (植物絵文字), `role="progressbar"` (進捗バー), `aria-label` 完全付与
- ⌨️ **キーボード操作**: フォーカス可視化、モーダル内 Tab トラップ、Enter で確定
- 🎨 **減らされた動き**: `@media (prefers-reduced-motion)` でアニメーション無効化対応済み

7. デプロイ・運用

Cloudflare Pages デプロイ手順

GitHub リポジトリ `ryotagtagtag-wq/schoolsync` を連携
Build command: `npm run build`
Output directory: `dist`
Root directory: `/` (サブディレクトリなし)
Deploy → `https://.pages.dev` で即公開
カスタムドメイン設定: `quest.ryopc.org`

ビルド成果物 (実測値)

`dist/index.html` : 0.94 kB (gzip: 0.60 kB)
`dist/assets/index-*.css` : 16.76 kB (gzip: 3.87 kB)
`dist/assets/index-*.js` : 157.27 kB (gzip: 51.12 kB)
ビルド時間: 約 700ms (Vite 5)
TypeScript 型チェック: `tsc` パス込み

環境変数・設定ファイル

```
# .env.example (本番では Cloudflare Pages 環境変数で設定)
VITE_APP_TITLE=Questra
VITE_APP_URL=https://quest.ryopc.org
# API キー等は不要 (localStorage 完結のため)
```

※ バックエンド・API キー・データベース接続情報は一切不要

8. 今後の拡張計画

短期 (v1.1～v1.2)

- 🌙 ダークモード対応 (OS 設定連動 + 手動切替)
- 🌐 多言語化 (i18n: 英語/中国語/韓国語)
- 📱 PWA 対応 (manifest.json, Service Worker, インストールプロンプト)
- 📊 週次/月次レポート (完了率推移グラフ、成長履歴)
- 📁 データエクスポート/インポート (JSON バックアップ)

中期 (v2.0)

- 🌱 新植物追加 (桜/盆栽/食虫植物/多肉/観葉植物...)
- 🐉 ドラゴン種類拡張 (属性別: 火/水/風/土/光/闇)
- 🎭 季節イベント (桜/夏祭り/紅葉/雪、限定植物・アイテム)
- 👤 フレンド機能 (匿名ID交換で相手の庭を見守り・応援)
- 🏆 実績/バッジシステム (連続日数、総タスク数、ドラゴン数等)

長期・構造的拡張

- 🔧 プラグイン API (独自クエストタイプ・アイテム・植物・ドラゴン定義)
- ☁️ クラウド同期オプション (任意・E2E暗号化、Firebase/Supabase 等)
- 🤖 AI 連携 (タスク提案、先生セリフ動的生成、成長予測)
- 📖 開発者向け SDK (外部アプリから Questra 状態参照/操作)
- 🎮 ミニゲーム要素 (植物と遊ぶ、ドラゴンと冒険)

技術的負債解消・品質向上

- ✅ 単体テスト導入 (Vitest + React Testing Library)
- ✅ E2E テスト (Playwright: 朝の挨拶→卵孵化→命名フロー)
- ✅ Storybook でコンポーネントカタログ化
- ✅ CI/CD パイプライン (GitHub Actions: lint → test → build → deploy)
- ✅ パフォーマンス監視 (Web Vitals, bundle size 監視)

Questra SPEC v1.0

作成者: **game_ryo** | Repository: [GitHub](#) | Live: <https://quest.ryopc.org>

Generated from source code analysis | Last updated: 2026-09-14